

Acceleration and Cryptid Structure of a Collatz-Like Machine

Compressing a 13-rule machine to a one-integer map — and the open halting problem at its core

1. Overview

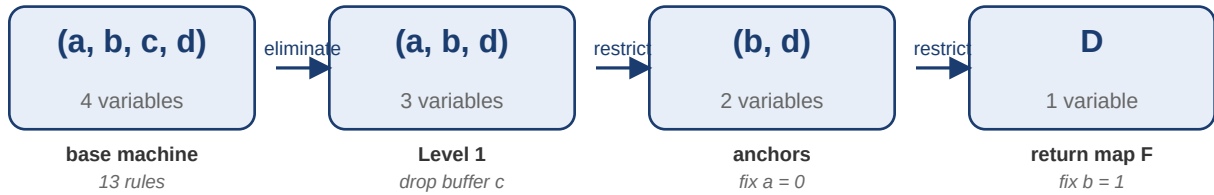
The subject of this report is the deterministic rewriting system $NextConfig(a, b, c, d)$: a machine whose **configuration** is a quadruple of non-negative integers and whose evolution is given by 13 **guarded rules**. A guarded rule is a statement “if the current configuration satisfies condition X (the *guard*), replace it by expression Y”; a rule whose guard holds is said to *fire*. The 13 guards are mutually exclusive and cover every configuration, so exactly one rule fires at each step and the system is deterministic. One rule is **HALT**: when its guard holds, the machine stops. The full rule set is reproduced in Section 2.

The report has two goals, and they are the same investigation seen from two ends. The first is to **accelerate** the machine: from most starts its counters grow geometrically, so a direct step-by-step simulation dies within a few thousand steps, and we replace it by an exactly equivalent but exponentially faster system. The second is to use that acceleration to **determine what the machine does** — in particular whether it halts from the all-zero start — and the answer is that it is a **cryptid**: a tiny, fully explicit program whose halting question is provably equivalent to an open problem of Collatz type. These goals are not independent. The acceleration is what makes the halting question *sayable*: only after compressing the machine to a single integer can one see the map, the exceptional set, and the pseudorandom orbit whose analysis reveals the cryptid. The hierarchy does not stall short of an answer; it runs until its next step and the halting question become literally the same object.

The engine of the acceleration is **batching**: when the same rule (or a fixed sequence of rules) provably fires many times in a row, the whole run is replaced by a single **closed-form** jump — one formula evaluated in place of an iteration. Every accelerated rule carries an exact **step certificate** counting the base steps it replaces, so the equivalence is *step-exact*, not merely behavioral: the accelerated systems visit exactly the states the original machine visits, at exactly the claimed step numbers. Every technical claim about this machine was verified mechanically against the original rules (Section 10).

Part I builds a four-level acceleration hierarchy:

Level 1 — variable elimination	c is a pure transfer buffer	4 variables $\rightarrow$ 3
Level 2 — loop batching	single-rule loops replaced by closed forms	each rule $O(1)$ arithmetic
Level 3 — cascade batching	two-rule loops batched geometrically ($2^n, 4^n$)	$\sim 9\times$ fewer macro steps
Level 4 — start-specific reduction	from (0, 0, 0, 0) the machine is a one-integer return map $D \rightarrow F(D)$	$\sim 60\times$ faster deep runs



*Levels 2–3 batch rules along the way: each firing-run or two-rule cascade collapses to one closed-form jump
 (compressing steps, not variables — the 10^{60} -step simulations of the Overview)*

Figure 1. The variable compression. Level 1 eliminates the buffer c ; restricting to the states the orbit actually revisits — anchors ($a = 0$), then the section $b = 1$ — removes a and then b , leaving the single integer D whose map is F . Levels 2–3 do not remove variables but batch rules, turning astronomically long step-runs into single closed-form jumps.

To cover 10^{60} base steps from a typical start, the level-2 system needs about 8,700 of its own steps and the level-3 system about 1,000 — each of constant cost — so total simulation work is polylogarithmic in the base step count. **Part II** then turns the compressed machine on itself. It proves an exact halting criterion (Section 8.1), reduces halting from $(0, 0, 0, 0)$ to a single orbit-avoidance question (Section 8.3), certifies that the machine does not halt within its first $10^{150,514}$ steps (Section 8.4), and shows in Section 9 why the residual question — like the Collatz conjecture — is believed but not provable: the machine is a cryptid. Three closing sections (10–12) collect the verification, performance, and code.

Part I — Accelerating the machine

Four levels of exact reduction turn an intractable 13-rule machine into the iteration of one integer. Each rule of each level carries a certificate for the base steps it replaces, so equivalence is step-exact throughout.

2. The base system

The original system, exactly as specified — with each guard's conditions restated in variable order a, b, c, d for readability (a guard is a conjunction, so ordering does not change its meaning). Each configuration is (a, b, c, d) , four non-negative integers; the guards below are mutually exclusive (their order does not matter) and complete (every configuration satisfies exactly one). We give each rule two labels, used interchangeably throughout the report: a number R1–R13, and a mnemonic name:

#	Name	Next configuration	Guard
R1	transfer	$(a, b + 1, c - 1, d)$	$c > 0$
R2	recharge	$(a + 1, 0, 0, 2b + 2)$	$c = 0, d = 0$
R3	drain	$(a - 3, b + 2, 0, d - 1)$	$a \geq 3, c = 0, d > 0$
R4	two	$(0, 0, b + 1, d - 1)$	$a = 2, c = 0, d > 0$
R5	pump	$(1, 1, b + 1, d - 1)$	$a = 1, b > 0, c = 0, d > 0$

#	Name	Next configuration	Guard
R6	jump	$(0, 0, 0, d + 2)$	$a = 1, b = 0, c = 0, d > 0$
R7	seed	$(0, 0, 2, 5)$	$a = 0, b = 0, c = 0, d = 1$
R8	seed	$(0, 0, 2, 2b + 1)$	$a = 0, b > 0, c = 0, d = 1$
R9	expand	$(3d - 4, 1, 2, 2b - 2d + 3)$	$a = 0, b \geq d - 1, c = 0, d \geq 2$
R10	reset3	$(3b + 2, 0, 0, 6)$	$a = 0, c = 0, d \geq 2, d = b + 3$
R11	reset4	$(3b + 4, 0, 0, 4)$	$a = 0, c = 0, d \geq 2, d = b + 4$
R12	shrink	$(3b + 3, 2, 0, d - b - 5)$	$a = 0, c = 0, d \geq 2, d \geq b + 5$
R13	halt	HALT	$a = 0, c = 0, d \geq 2, d = b + 2$

The names describe what each rule does to the configuration: **transfer** moves one unit from c to b; **recharge** refills d (with $2b + 2$) when it is empty, resetting b and incrementing a; **drain** drains a, three at a time, into b, consuming d; **two** is the $a = 2$ case, handing b onward through c; **pump** pumps b up while d ticks down; **jump** jumps d up by 2; **seed** seeds a fresh d from b when $d = 1$ (two guard cases, same role); **expand** expands a to about $3d$ and d to about $2b$; **reset3/reset4** reset b to 0 with a small fixed d; **shrink** shrinks d by $b + 5$ while tripling b into a.

A **trajectory** is the sequence of configurations produced by firing rules repeatedly; the machine **halts** if it ever reaches a configuration satisfying R13's guard. Note the scale of the problem: from most starts the values grow without bound (Section 7), so direct simulation quickly becomes impossible — from $(0, 0, 0, 0)$ the values are astronomically large within a few thousand of the accelerated steps defined below.

3. Three structural observations

3.1 The variable c is a transfer buffer (Level 1)

R1 (transfer) is the only rule that fires when $c > 0$, and it simply moves one unit from c to b. Applying it c times in a row telescopes:

$$(a, b, c, d) \rightarrow (a, b + 1, c - 1, d) \rightarrow \dots \rightarrow (a, b + c, 0, d) \quad [c \text{ base steps}],$$

and every rule that writes a positive c (R4, R5, R7, R8, R9) is immediately followed by exactly this transfer, because no other rule can fire while $c > 0$. So c never carries information: the system is fully described on triples via the projection $(a, b, c, d) \rightarrow (a, b + c, d)$. This eliminates one variable and the transfer rule outright — that is Level 1.

3.2 Two rules are unary loops

Call a rule a **unary loop** if firing it re-establishes its own guard, so the *same* rule fires again and again until a counter runs out (“unary” because the loop body is a single rule; Level 3 will handle loops whose body is a fixed *sequence* of rules). A unary loop is what makes batching possible: the whole run of consecutive firings is a predictable arithmetic progression, so it collapses to one formula.

Why R3 (drain) is a unary loop. R3's guard is $a \geq 3$, $c = 0$, $d > 0$, and its update adds $(-3, +2, 0, -1)$ to (a, b, c, d) . Two things follow. First, the update leaves $c = 0$, so no other rule can interpose. Second, the update moves only a and d toward their guard boundaries, by fixed amounts — so whether the guard still holds after a firing depends only on how far a and d were from those boundaries. Concretely, after j consecutive firings the configuration is, by induction,

$$(a, b, 0, d) \rightarrow (a - 3j, b + 2j, 0, d - j),$$

and firing number $j + 1$ is allowed exactly when $a - 3j \geq 3$ and $d - j > 0$ — equivalently $j + 1 \leq \text{floor}(a/3)$ and $j + 1 \leq d$. So the run has length exactly

$$k = \min(\text{floor}(a/3), d),$$

and the entire run collapses to the single jump $(a, b, d) \rightarrow (a - 3k, b + 2k, d - k)$, with step certificate k . Two small examples: from $(11, 0, 2)$: $(11, 0, 2) \rightarrow (8, 2, 1) \rightarrow (5, 4, 0)$, stopped by d hitting 0, and indeed $k = \min(\text{floor}(11/3), 2) = \min(3, 2) = 2$. From $(7, 0, 5)$: $(7, 0, 5) \rightarrow (4, 2, 4) \rightarrow (1, 4, 3)$, stopped by a dropping below 3, and $k = \min(2, 5) = 2$.

R5 (pump) is a loop one level up. R5 alone does not re-establish its own guard (it sets $c = b + 1 > 0$), but the $c > 0$ it creates forces exactly $b + 1$ firings of R1, after which the configuration has returned to the original *shape*:

$$(1, b, 0, d) \xrightarrow{R5} (1, 1, b + 1, d - 1) \xrightarrow{R1 \times (b+1)} (1, b + 2, 0, d - 1),$$

— same $a = 1$, b grown by 2 (so still positive), same $c = 0$, d shrunk by 1. The loop body is thus the fixed sequence $(R5, R1 \times (b+1))$, and only d moves toward a boundary, so the loop runs exactly d rounds. Section 4.1 turns this into the pump rule and sums its cost. Batching these two loops is the heart of Level 2.

3.3 The long-run dynamics is a linear map

Empirically, about 85% of all level-2 steps consist of one particular two-rule alternation: R12 (shrink) followed by a full drain run. Why does that run return to $a = 0$? Because R12 outputs $a = 3b + 3 = 3(b + 1)$, a multiple of 3 — so the drain loop of Section 3.2, which subtracts 3 per firing, can consume a completely (when d lasts long enough, which the guard $d \geq 2b + 6$ guarantees; Section 5.1 does the arithmetic). The composition is the **affine map** (a linear map plus constants)

$$(b, d) \rightarrow (2b + 4, d - 2b - 6) \quad \text{while } d \geq 2b + 6,$$

and iterating an affine map n times has a geometric closed form (powers of 2 appear because b is doubled each round). Batching such two-rule loops — we call them **cascades** — is Level 3.

4. The compressed system (Level 2)

The level-2 system acts on triples (a, b, d) ; an original configuration embeds as $(a, b, c, d) \rightarrow (a, b + c, d)$. We call its states **macro states** and its rule applications **macro steps**; each macro step bundles one or more base steps. The guards below are, like the base table's, mutually exclusive and complete, so order does not matter (provable by a short case analysis of d against b — the partition lemma of Section 6.1 shows the pattern — and machine-checked on 60,000 states); we list the rules in base order with halt last, exactly as in Section 2. Each rule keeps the **index and name** of the base

rule it accelerates (new rules at later levels will receive fresh indices R14, R15, ...). The marker after the name records the rule's relation to the previous level, in a convention used for every table from here on: • = carried over verbatim (here: the identical base rule, with c dropped); **(batch)** = the same rule, all consecutive firings collapsed into one jump; **(fuse)** = the rule together with the forced steps that always follow it. The **Built from** column gives the exact base composition, and the last column the step certificate:

#	Name	Next macro state	Guard	Built from	Steps
R2	recharge •	$(a + 1, 0, 2b + 2)$	$d = 0$	R2	1
R3	drain (batch)	$(a - 3k, b + 2k, d - k),$ $k = \min(\text{floor}(a/3), d)$	$a \geq 3, d > 0$	$R3 \times k$	k
R4	two (fuse)	$(0, b + 1, d - 1)$	$a = 2, d > 0$	$R4, R1 \times (b+1)$	$b + 2$
R5	pump (batch+fuse)	$(2, 0, 2b + 4d + 2)$	$a = 1, b > 0, d > 0$	$(R5, R1 \times \cdot) \times d,$ R2	$bd + d^2 + d + 1$
R6	jump •	$(0, 0, d + 2)$	$a = 1, b = 0, d > 0$	R6	1
R7/8	seed (fuse)	$(0, 2, 2b + 1); (0, 2, 5) \text{ if } b = 0$	$a = 0, d = 1$	$R8/R7, R1 \times 2$	3
R9	expand (fuse)	$(3d - 4, 3, 2b - 2d + 3)$	$a = 0, b \geq d - 1, d \geq 2$	$R9, R1 \times 2$	3
R10	reset3 •	$(3b + 2, 0, 6)$	$a = 0, d = b + 3, d \geq 2$	R10	1
R11	reset4 •	$(3b + 4, 0, 4)$	$a = 0, d = b + 4, d \geq 2$	R11	1
R12	shrink •	$(3b + 3, 2, d - b - 5)$	$a = 0, d \geq b + 5, d \geq 2$	R12	1
R13	halt •	HALT	$a = 0, d = b + 2, d \geq 2$	R13	—

4.1 Where each level-2 rule comes from

The • rows (halt, recharge, jump, reset3, reset4, shrink) are single base rules that neither read nor write c — they pass to the triple form unchanged. The others are compositions, with c eliminated by Section 3.1:

two = R4 then the c-transfer: $(2, b, 0, d) \rightarrow (0, 0, b + 1, d - 1) \xrightarrow{R1 \times (b+1)} (0, b + 1, 0, d - 1)$; total $1 + (b + 1) = b + 2$ base steps. **seed** and **expand** are R7/R8 and R9 followed by their two c-transfers (3 steps each).

drain = R3 iterated. Each firing does $(a, b, d) \rightarrow (a - 3, b + 2, d - 1)$ and R3's guard keeps holding until $a < 3$ or $d = 0$, i.e. for exactly $k = \min(\text{floor}(a/3), d)$ firings; summing gives $(a - 3k, b + 2k, d - k)$ in k steps.

pump = d rounds of (R5 + transfers), then R2. One round: $(1, b, 0, d) \xrightarrow{R5} (1, 1, b + 1, d - 1) \xrightarrow{R1 \times (b+1)} (1, b + 2, 0, d - 1)$ — net effect: b becomes $b + 2$, d becomes $d - 1$, at cost $b + 2$. After d rounds the state is $(1, b + 2d, 0, 0)$; R2 (recharge) then always fires, giving $(2, 0, 0, 2(b + 2d) + 2) = (2, 0, 0, 2b + 4d + 2)$. Total cost $\sum_{i < d} (b + 2i + 2) + 1 = bd + d^2 + d + 1$. Because the recharge is forced, it is fused into the pump rule.

Compared with the original: one variable and one rule eliminated, both loops replaced by closed forms, and every rule now costs a constant amount of integer arithmetic regardless of the values involved.

5. Cascade super-rules (Level 3)

Two *compositions* of level-2 rules are themselves loops: each composition returns to a state where the same composition applies again. We write $(X; Y)^n$ for n consecutive rounds of firing rule X then rule Y ; such iterates have geometric closed forms. In both cases n is the largest round count whose guard still holds; because one quantity grows geometrically while the comparison side moves slowly, n is found by binary search over a monotone inequality.

5.1 Cascade A: shrink–drain (~85% of all macro steps)

One round, derived from the level-2 table: from $(0, b, d)$ with $d \geq 2b + 6$, shrink gives $(3b + 3, 2, d - b - 5)$. Now $\text{floor}((3b + 3)/3) = b + 1$ and the guard implies $d - b - 5 \geq b + 1$, so the drain runs its full $k = b + 1$ and lands back on $a = 0$:

$$(0, b, d) \xrightarrow{\text{shrink}} (3b + 3, 2, d - b - 5) \xrightarrow{\text{drain}} (0, 2 + 2(b + 1), d - b - 5 - (b + 1)) = (0, 2b + 4, d - 2b - 6),$$

which is the affine map of Section 3.3, at cost $(b + 2)$ base steps per round. Iterating while the guard holds — b doubles-plus-4 each round — and summing the geometric series:

$$b_n = 2^n(b + 4) - 4, \quad d_n = d - 2(b + 4)(2^n - 1) + 2n, \quad \text{cost} = (b + 4)(2^n - 1) - 2n \text{ base steps.}$$

(To check the first: $b_1 = 2(b + 4) - 4 = 2b + 4$. The $2n$ term in d_n collects the +4-then-doubled constants.) The batched rule — all n rounds in one jump — is named **cascadeA**.

5.2 Cascade B: sweep (full-drain + recharge)

When $a \geq 3d$, the drain's $k = \min(\text{floor}(a/3), d)$ equals d , so the drain exhausts d completely and recharge is forced next:

$$(a, b, d) \xrightarrow{\text{drain}} (a - 3d, b + 2d, 0) \xrightarrow{\text{recharge}} (a - 3d + 1, 0, 2b + 4d + 2).$$

Here d roughly quadruples per round while a shrinks, so a cascade has at most $\sim \log_4(a)$ rounds. The first round zeroes b , so later rounds are the $b = 0$ case; writing $w = 6b + 12d + 8$ (three times the d -value after round 1, plus 2) and $T_j = (4^j - 1)/3$, the state after n rounds is:

$$a_n = a - 3d + 3n - 2 - wT_{n-1}, \quad d_n = (w \cdot 4^{n-1} - 2)/3 \text{ — check } n = 1: (a - 3d + 1, 0, 2b + 4d + 2) \text{ as above; these match the sweep row of the table below exactly.}$$

The batched rule is named **sweep**.

5.3 The complete level-3 system in closed form

The full accelerated system, with both cascades folded in, is again a 13-rule guarded system — the same size as the original, but with every rule a closed-form expression. The sweep captures every state with $a \geq 3d$, so the drain that remains carries the conjunct $a < 3d$, under which its k simplifies to $\text{floor}(a/3)$. Abbreviations:

$$q = \text{floor}(a/3), \quad w = 6b + 12d + 8, \quad T_j = (4^j - 1)/3,$$

$$n_A = \max\{ m \geq 1 : (b + 4) \cdot 2^{m+1} \leq d + 2(b + 4) + 2m \},$$

$$n_B = \max\{ m \geq 1 : w \cdot 4^{m-1} \leq 3(a - 3d) + w + 9(m - 1) \}.$$

n_A and n_B are the numbers of rounds the two cascades run before their guards fail: the inequalities say “round m is still allowed”, and since each is monotone in m the maxima are computable by binary search (up to ± 1 they equal $\text{floor}(\log_2((d + 2b + 8)/(2b + 8)))$ and $\text{floor}(\log_4(3(a - 3d)/w)) + 1$). In the table, n stands for n_A or n_B as appropriate. Guards are again mutually exclusive (order-free), listed in base order with halt last; rules keep their base indices and the two new rules receive the fresh indices R14 and R15, placed beside the rules they grew out of. Markers as in Section 4 ($\bullet$ = carried over from Level 2 verbatim), plus two more: **(new)** = a batched two-rule composition that exists only from this level on, and **(mod.)** = a Level-2 rule whose formula or guard changed here — the changed conjunct is visible in the guard. The **Built from** column gives each rule's level-2 origin: for modified rules, which part of the old guard survives; for new rules, the batched composition:

#	Name	Next macro state	Guard	Built from	Base steps
R2	recharge $\bullet$	$(a + 1, 0, 2b + 2)$	$d = 0$	R2	1
R3	drain (mod.)	$(a \bmod 3, b + 2q, d - q)$	$a \geq 3, a < 3d, d > 0$	R3 ($a < 3d$ part)	q
R14	sweep (new)	$(a - 3d + 3n - 2 - wT_{n-1}, 0,$ $(w \cdot 4^{n-1} - 2)/3)$	$a \geq 3, a \geq 3d, d > 0$	$(R3; R2)^n$	$d + 1 +$ $(wT_{n-1} + n - 1)/3$
R4	two $\bullet$	$(0, b + 1, d - 1)$	$a = 2, d > 0$	R4	$b + 2$
R5	pump $\bullet$	$(2, 0, 2b + 4d + 2)$	$a = 1, b > 0, d > 0$	R5	$bd + d^2 + d + 1$
R6	jump $\bullet$	$(0, 0, d + 2)$	$a = 1, b = 0, d > 0$	R6	1
R7/8	seed $\bullet$	$(0, 2, 2b + 1); (0, 2, 5) \text{ if } b = 0$	$a = 0, d = 1$	R7/8	3
R9	expand $\bullet$	$(3d - 4, 3, 2b - 2d + 3)$	$a = 0, b \geq d - 1, d \geq 2$	R9	3
R10	reset3 $\bullet$	$(3b + 2, 0, 6)$	$a = 0, d = b + 3, d \geq 2$	R10	1
R11	reset4 $\bullet$	$(3b + 4, 0, 4)$	$a = 0, d = b + 4, d \geq 2$	R11	1
R12	shrink (mod.)	$(3b + 3, 2, d - b - 5)$	$a = 0, b + 5 \leq d \leq 2b + 5$	R12 (band part)	1
R15	cascadeA (new)	$(0, 2^n(b + 4) - 4, d - 2(b + 4)(2^n - 1) + 2n)$	$a = 0, d \geq 2b + 6$	$(R12; R3)^n$	$(b + 4)(2^n - 1) - 2n$
R13	halt $\bullet$	HALT	$a = 0, d = b + 2, d \geq 2$	R13	—

Exactly four rows differ from Level 2. **sweep** is new: it is the batched loop $(\text{drain}; \text{recharge})^n = (R3; R2)^n$ of Section 5.2. **cascadeA** is new: the batched loop $(\text{shrink}; \text{drain})^n = (R12; R3)^n$ of Section 5.1. **drain** (R3) is modified: only its $a < 3d$ part survives (sweep takes the rest), and under that conjunct $k = \min(\text{floor}(a/3), d)$ simplifies to $q = \text{floor}(a/3)$. **shrink** (R12) is modified only in its guard: cascadeA

(R15) captures $d \geq 2b + 6$, so shrink's guard narrows from $d \geq b + 5$ to the band $b + 5 \leq d \leq 2b + 5$ where cascadeA runs zero rounds; its formula and cost are untouched. Every other rule (halt, recharge, two, pump, jump, seed, expand, reset3, reset4) carries over verbatim. The closed forms — round counts n , resulting states, and base-step costs — were verified against loop implementations on 50,000 random states per cascade (round counts up to $n = 46$), and the assembled system on a further 20,000 random states and full trajectories.

6. Level 4: the one-dimensional system from (0, 0, 0, 0)

When the start is fixed at (0, 0, 0, 0), the system compresses one level further — to a single integer. This section is the hinge of the report: it completes the acceleration (Part I) and hands Part II a one-variable map to interrogate. Because so much rests on it, it separates definitions, proofs, and verified observations explicitly. Roadmap: **6.1** defines anchors, shows that the whole dynamics reduces to an anchor-to-anchor map G given by a finite closed-form table (including exactly what happens *between* anchors), then defines the one-variable return map F and states precisely which parts of that definition are proved and which are verified. **6.2** defines branch words and proves that F is piecewise affine with countably many pieces — the exact sense in which F has no closed form. **6.3** explains the runner's algorithmics and why it sits at the complexity floor. **6.4** assembles the final 11-rule system.

The logical structure of the reduction deserves emphasis, because its parts have different statuses. Reduction to anchors is a *theorem* (6.1). Reduction to one variable rests on the orbit returning to $b = 1$ each cycle — a property we can verify massively but, as 6.1 explains, cannot prove. Section 6.1 therefore closes with a case-split proposition: exactly one of three things can happen to the machine.

The halting set H , in plain terms. Once the machine is a map $D \rightarrow F(D)$ on a single integer, its whole fate hinges on which values of D lead, sooner or later, to the HALT rule. Collect those values into one set:

$$H = \{ D : \text{starting a super-cycle from } D \text{ eventually halts} \}.$$

H is small and scattered — the smallest members are 2, 3, 9, 16, 19, ... (from $D = 2$, for instance, one super-cycle runs $(1, 2) \rightarrow (4, 0) \rightarrow (0, 12) \rightarrow (4, 6) \rightarrow \text{HALT}$). The machine starts at $D = 17$ and each super-cycle jumps it to a new value $F(D)$; halting from (0, 0, 0, 0) is nothing more than the question *does this jumping ever land on a value in H ?* That single question is what the rest of the report circles: 6.2–6.4 build the machinery of F , Section 8 pins down H exactly and searches for a landing, and Section 9 explains why — like the Collatz problem — no one can currently prove that it never does. It is worth saying at the outset where each of the three cases is dealt with:

Case	What it means	Where it is handled
(i) some super-cycle halts	the machine halts; the D_k opening that cycle lies in the halting set H	studied in Section 8; verified never to occur in the first 200,000 cycles
(ii) every super-cycle returns	F is total on the orbit; the machine halts iff the F -orbit ever meets H	the working hypothesis of 6.2–6.4 (structure and runner of F); the open question of Sections 8–9

(iii) some super-cycle never returns	the machine runs forever inside that cycle	fully closed in 6.1: implies non-halting outright; never observed
--------------------------------------	--	--

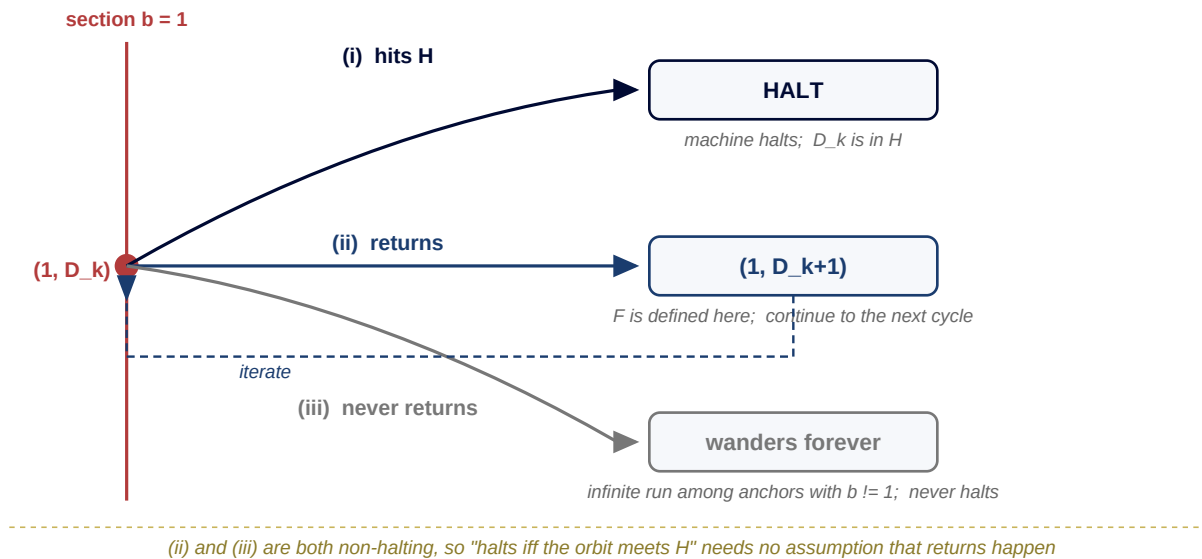


Figure 2. The three fates of a super-cycle, and why the reduction survives the one gap in it. Only case (i) halts; cases (ii) and (iii) are both infinite runs. So “the machine halts iff the orbit meets H” holds whether or not returns to the section can be guaranteed — a failure to return would itself certify non-halting.

So the only case requiring further work is (i) versus (ii) — does the F-orbit meet H? — and that question is insensitive to the unproved return property: whichever way case (iii) could have gone, “the machine halts iff the F-orbit meets H” stands, and a failure to return would itself certify non-halting. Everything after 6.1 develops the tools to pursue that one question.

The motivating observation, from direct simulation: the trajectory reaches $(0, 1, 0, 17)$ at base step 25, and from then on repeatedly passes through states of the form $(0, 1, 0, D)$:

$D: 17, 57, 185, 601, 2265, 5913, 16121, 35177, 71833, \dots$

Because the system is deterministic and three of the four coordinates are pinned ($a = 0, b = 1, c = 0$), the entire future from such a state is a function of D alone — two visits with the same D have identical futures. We call the stretch from one such state to the next a **super-cycle**, and write $F(D)$ for the value of d at the next such state. The **F-orbit of 17** is the resulting sequence $D_0 = 17, D_{k+1} = F(D_k)$; D_k denotes the value after k super-cycles. Whether the machine is *guaranteed* to keep returning to $b = 1$ states is a fair question; Section 6.1 addresses it head-on.

6.1 Formalization: the anchor map G

Definition. An **anchor** is a macro state with $a = 0$ ($c = 0$ always holds at this level); we write it as the pair (b, d) . Anchors are the right waypoints because the halt state is an anchor, and because the dynamics keeps producing them, as we now show.

Anchors live on $a = 0$; every rule chain leaves the plane ($a > 0$) and lands back on it
 one super-cycle = the excursions from one $b = 1$ anchor to the next $\rightarrow D_{k+1} = F(D_k)$

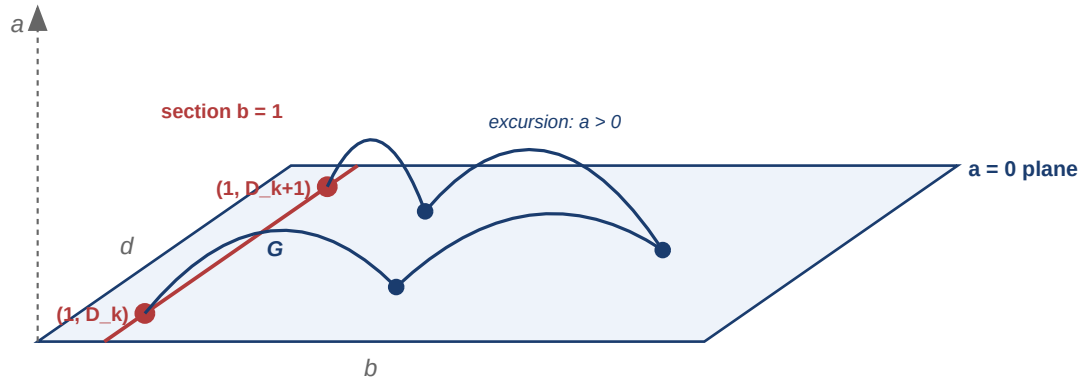


Figure 3. Anchors are the states lying in the $a = 0$ plane. Every rule chain lifts the state off that plane ($a > 0$) and provably lands back on it, so the dynamics is a walk from anchor to anchor — the map G . The section $b = 1$ is a line inside the plane; the excursions between two of its points make up one super-cycle, and the map it induces on the remaining coordinate is F .

What happens between anchors. Start at any anchor. Exactly one row of the table below applies — that is the partition lemma, proved just below. Each row summarizes a stretch of ordinary level-2/3 execution, and its **Built from** column spells that stretch out: it lists, in order, the level-2/3 rules that fire between this anchor and the next one. For example, row R17's entry "R9; R3; R4" means: from any anchor satisfying R17's guard, the machine fires expand, then drain, then two, and is then back at an anchor.

Two facts make this a meaningful summary. *The chain is fixed:* although the machine simply fires whichever rule's guard holds, each row's guard provably forces the *same* rule sequence for every anchor satisfying it — each intermediate state satisfies the guard of the next rule in the listed chain (this is the content of the row derivations; the worked example below traces one in full). The batched rules inside a chain may run for zero or more iterations, with counts determined by the entering values, but the pattern of rules never varies. *No anchor is skipped:* an anchor is a state with $a = 0$, and every intermediate state of these chains has $a > 0$ (including the $d = 0$ states, which occur only with $a > 0$ there) — so the chain's endpoint is the *first* anchor after the start, and $G(b, d)$ genuinely is the next-anchor map. The one exception proves the rule: inside a SWEEP, a cascade-B round can land exactly on a $a = 0$ mid-chain (the $A' = 3\Delta'$ equality described below); the SWEEP definition detects this case ($A^* = 1$) and stops G at that interior anchor, preserving the property. That anchors always lead to anchors is worth stating as a theorem:

Lemma (the guards partition the plane). For every pair (b, d) of non-negative integers, exactly one of the twelve guards in the table holds.

Proof. If $d = 0$ or $d = 1$, the first two rows apply and no other guard can hold: the remaining guards all force $d \geq 2$ — the explicit conjuncts do so directly; $3d = 2b + 5$ has no solution with $d \leq 1$ and $b \geq 0$; $3d$

$> 2b + 5$ with $d \leq 1$ would force $b < 0$; and the rows on and beyond the line $d = b + 2$ have $d \geq 2$ outright. Now let $d \geq 2$ and compare d with b ; exactly one of four cases holds. **(1)** $d \leq b + 1$: the rows requiring $d \geq b + 2$ are excluded, and the trichotomy of $3d$ against $2b + 5$ selects exactly one of R17 (below), R18 (equal), or the pair R19/R20 (above), the last two split by the dichotomy of $11d$ against $10b + 24$. Conversely, $3d \leq 2b + 5$ with $d \geq 2$ already implies $d \leq b + 1$ (since $d \geq b + 2$ gives $3d \geq 3b + 6 > 2b + 5$), which is why R17 and R18 need not carry that conjunct. **(2)** $d \in \{b + 2, b + 3, b + 4\}$: exactly one of halt, R21, R22 — and every other guard fails there: $3d \geq 3b + 6 > 2b + 5$ excludes R17/R18, $d > b + 1$ excludes R19/R20, $d < b + 5$ excludes R23/R24, and $d \geq 2b + 6$ would force $b \leq -2$. **(3)** $b + 5 \leq d \leq 2b + 5$: the dichotomy of $15d$ against $18b + 61$ selects exactly one of R23, R24; all other guards fail as in case (2). **(4)** $d \geq 2b + 6$: only R25 holds. The four cases cover every $d \geq 2$. **QED** (The lemma is also machine-checked on 60,000 random pairs.)

Theorem (anchor recurrence). From every anchor other than the halt state, the machine reaches another anchor after finitely many macro steps: at most 6 for the non-SWEEP rows, and at most $2n_B + 6$ for the SWEEP rows. Consequently every trajectory that starts at an anchor either halts or visits anchors forever, and the anchor map $G(b, d) = \text{next anchor}$ captures the dynamics completely.

Proof. The partition lemma gives exactly one applicable row. For the eight non-SWEEP rows, the Built-from chain is an explicit composition of at most six level-2 rules whose intermediate states all have a > 0 (the worked example below traces one), and each such chain ends on a $= 0$ by the row's derivation — verified against composed level-2 steps on 40,000 random anchors. A SWEEP row runs its prefix of at most three rules, then cascade B: each round more than quadruples the d -value ($d \rightarrow 4d + 2$) while a decreases, so the round guard $a \geq 3d$ fails after at most $\log_4 a$ rounds — n_B is finite — and the exit chain of at most three further rules again ends on a $= 0$. Every case lands on an anchor after finitely many macro steps. **QED**

Table conventions: guards are mutually exclusive (order-free), listed with halt last; seed and halt keep their base indices, and the new anchor-level compositions receive fresh indices R16–R25, ordered by the base index of their leading rule. The Base steps column is the exact certificate; $P(\beta, \delta) = \beta\delta + \delta^2 + \delta + 1$ abbreviates the pump certificate, and “+ sweep” adds the certificate of the SWEEP continuation (closed forms in `afterstep25.py`):

#	Name	Next anchor $G(b, d)$	Guard	Built from	Base steps
R16	recharge+	$(0, 2b + 4)$	$d = 0$	R2; R6 (recharge; jump)	2
R7/8	seed •	$(2, 2b + 1); (2, 5) \text{ if } b = 0$	$d = 1$	R7/8 (seed)	3
R17	expand+	$(2d, 2b - 3d + 4)$	$d \geq 2, 3d < 2b + 5$	R9; R3; R4 (expand; drain; two)	$3d + 2$
R18	boundary	$(2, 4d - 1)$	$3d = 2b + 5$	R9; R3; R2; R3 (a, d hit 2, 0 together)	$d + 3$
R19	expand-deep	$(6d - 4b - 8, 10b - 11d + 24)$	$d \leq b + 1, 3d > 2b + 5,$ $11d \leq 10b + 24$	R9; R3; R2; R3 (expand; drain; recharge; drain)	$d + 3$

#	Name	Next anchor $G(b, d)$	Guard	Built from	Base steps
R20	expand-sweep	$SWEEP(9d - 6b - 12, 8b - 8d + 20)$	$d \leq b + 1, 11d > 10b + 24$	R9; R3; R2 (expand; drain; recharge)	$2b - 2d + 7 + \text{sweep}$
R21	reset3+	$SWEEP(3b + 2, 6)$	$d = b + 3$	R10 (reset3)	$1 + \text{sweep}$
R22	reset4+	$SWEEP(3b + 4, 4)$	$d = b + 4$	R11 (reset4)	$1 + \text{sweep}$
R23	exit	$(1, 16(d - b) - 55)$	$b + 5 \leq d \leq 2b + 5, 15d > 18b + 61$	R12; R3; R2; R3; R5; R4 (shrink; drain; recharge; drain; pump; two)	$b + 5 + P(4b - 2d + 12, 5d - 6b - 20)$
R24	shrink-sweep	$SWEEP(6b - 3d + 19, 4d - 4b - 14)$	$b + 5 \leq d \leq 2b + 5, 15d \leq 18b + 61$	R12; R3; R2 (shrink; drain; recharge)	$d - b - 3 + \text{sweep}$
R25	round	$(2b + 4, d - 2b - 6)$	$d \geq 2b + 6$	R12; R3 (shrink; drain): one R15 round	$b + 2$
R13	halt •	HALT	$d = b + 2$	R13 (halt)	—

$SWEEP(A, \Delta)$ abbreviates the continuation from the state $(A, 0, \Delta)$: cascade B runs its n_B closed-form rounds to the exit $(A^*, 0, \Delta^*)$; $A^* = 1$ signals that the last round began at values A', Δ' with $A' = 3\Delta'$ exactly, so G stops at the interior anchor $((\Delta^* - 2)/2, 0)$; otherwise the exit drains fully with $q = \text{floor}(A^*/3)$ and, by $A^* \bmod 3$, the next anchor is $(2q, \Delta^* - q)$, or $(2q + 1, \Delta^* - q - 1)$, or — via pump — $(1, 4\Delta^* + 1)$, a second F-exit. Naming: seed and halt carry over from Level 2 unchanged (•), merely restated on anchors; round is a single cascadeA round left un-batched (each round is an anchor step here); every other row is a new anchor-level composition.

Worked example (one anchor step). Take row R17 from an anchor $(0, b, d)$ with $d \geq 2$ and $3d < 2b + 5$. Its chain runs: expand to $(3d - 4, 3, 2b - 2d + 3)$; drain — running its full $k = \text{floor}(a/3) = d - 2$, which the guard $3d < 2b + 5$ keeps below the current d -value $2b - 2d + 3$ — to $(2, 2d - 1, 2b - 3d + 5)$; two to $(0, 2d, 2b - 3d + 4)$ — an anchor again, three macro rules and $3d + 2$ base steps later, with every intermediate state having $a > 0$. Every row of the table compresses such a chain.

Definition (the return map). The section $\{b = 1\}$ is the set of anchors whose b -component equals 1. F is the **first-return map** of G to this section: from the anchor $(1, D)$, apply G repeatedly until the b -component is again 1; $F(D)$ is the d -component of that anchor. The section is the natural finishing line because only three of the twelve outcomes can output $b' = 1$ — exit (R23) and two of SWEEP's exit cases — and each of them is the final, pump-driven step of a growth phase.

Is the return to $b = 1$ guaranteed? The theorem guarantees an endless anchor sequence; the open part is whether it must revisit $b = 1$. A reformulation sharpens the question: inspecting the twelve rows, a $b' = 1$ anchor arises in exactly three ways — the exit row and SWEEP's pump exit (both contain a pump firing), and SWEEP's $A^* = 2$ exit (a two firing) — and conversely every pump firing yields a $b = 1$ anchor two macro steps later. So guaranteed return says: the anchor dynamics can never avoid these three outcomes forever.

The evidence is strong and uniform: every anchor with $b, d \leq 400$ (160,000+ states) and over 9,000 random anchors with coordinates up to 320 bits reached the section or halted, never taking more than

80 anchor steps, and the observed maximum shows no growth with scale (52, 43, 41, 50, 45, 69, 48 at 10–320 bits); this check is now part of formal.py's verification suite. We nevertheless do *not* state guaranteed return as a theorem, for two principled reasons. First, **return time is provably unbounded**, so no finite case analysis can settle the question: the expand+ row acts on the ratio $x = d/b$ as $x \rightarrow (2 - 3x)/(2x)$, which has a repelling fixed point at $x = 1/2$, and anchors constructed near the line $b = 2d$ wander for $\Theta(\log d)$ steps before exiting — measured expand+ chain lengths 5, 9, 19, 39, 79 at $d = 2^{10}, 2^{20}, 2^{40}, 2^{80}, 2^{160}$. Second, in the same ratio coordinate the tail rows form an **expanding piecewise interval map**, and returning means eventually landing in fixed exit windows; orbits of expanding maps that avoid fixed windows forever form a thin, Cantor-like, measure-zero set — a set sampling cannot rule out, with exactly the thin-set-avoidance character of the halting question itself (Section 9). Even this bookkeeping lemma, in other words, is Collatz-adjacent.

Proposition (the reduction is unconditional). Started at $(0, 0, 0, 0)$, exactly one of the following holds. **(i)** Some super-cycle ends in the halt anchor; then the machine halts, and the value D_k opening that cycle lies in the halting set H of Section 8.3. **(ii)** Every super-cycle returns; then the F -orbit $D_0, D_1, \dots$ is infinite, avoids H , and the machine never halts. **(iii)** Some super-cycle neither halts nor returns; then the machine runs forever inside that cycle — an endless wander among anchors with $b \neq 1$ — and again never halts. In particular, **the machine halts if and only if the F -orbit reaches H** , with no assumption that returns are guaranteed; a failure of return, far from undermining the analysis, would itself certify non-halting.

Proof. By the recurrence theorem, the trajectory from the anchor $(1, 17)$ is an infinite sequence of anchors unless it reaches the halt anchor. List its (possibly finite) visits to the section $b = 1$; these open super-cycles with values $D_0 = 17, D_1, \dots$. If the halt anchor is reached, it is reached inside some cycle k , which is precisely the statement $D_k \in H$ — case (i). Otherwise the anchor sequence is infinite: if it visits the section infinitely often, case (ii); if only finitely often, case (iii). The three cases are exhaustive and mutually exclusive, and in (ii) and (iii) the machine, running through infinitely many anchors, never halts. **QED**

The proposition is also why the verification architecture never relies on returns: the halting criterion of Section 8 is checked at *every anchor*, so the section is bookkeeping — it delimits super-cycles and defines F — not an assumption. A hypothetical non-returning run would appear as one unboundedly long, still fully checked super-cycle, never as an unsound conclusion. On the actual orbit of 17, all 200,000 verified super-cycles returned, with median 3 and maximum 42 anchor steps. Verification of this subsection: the table against composed level-2 steps on 40,000 random anchors across ten orders of magnitude; the induced F against the level-4 runner for 800 orbit cycles; section return on the 40,401-anchor box plus 2,000 scale-spread random anchors (formal.py).

6.2 Branch words; why F is piecewise affine, not a single formula

Definition (branch word). Fix a super-cycle starting at $(1, D)$, and let $r_1, r_2, \dots, r_L$ be the indices of the G -rows fired, in order, until the step that lands back on $b = 1$. The **branch word** $w(D)$ is this sequence — enriched, for each SWEEP row, with its cascade- B round count n_B and exit case, which the row index alone does not determine. The **word length** $|w(D)| = L$ is the number of anchor steps in the cycle. Because G is deterministic, $w(D)$ and $F(D)$ are functions of D .

Why word lengths grow along the orbit. From $(1, D)$ with $D \geq 8$, the round row R_{25} applies (its guard $d \geq 2b + 6$ reads $D \geq 8$ here), and by the closed forms of Section 5.1 with $b + 4 = 5$ it keeps

applying for exactly $n_A(1, D)$ consecutive anchor steps. Hence $|w(D)| \geq n_A(1, D)$, and by the logarithm formula of Section 5.3, $n_A(1, D) \approx \log_2(D/10)$. Along the orbit, D_k increases strictly ($F(D) > D$ on every verified cycle; minimum observed ratio 1.0002, average $\times 2.4$), reaching 250,005 bits by cycle 200,000 — so cycle k 's word has roughly 1.25k letters and word lengths grow without bound. Measured directly: at least 865 distinct words occur in the first 3,000 cycles.

Proposition (piecewise affinity). For each word w there are integers α_w, β_w with $F(D) = \alpha_w D + \beta_w$ for every D whose branch word is w . *Proof.* Fixing w fixes, at each of the L anchor steps, which row fires and (for SWEEP rows) its round count and exit case; each step is then one fixed affine map of (b, d) , and a composition of affine maps is affine; restricting to the start $(1, D)$ leaves an affine function of D alone. **QED** Moreover the set $\{D : w(D) = w\}$ is cut out by finitely many linear inequalities and congruences — the guards met along the way, translated back to D — so each piece is explicitly definable.

Why countably many pieces, not finitely many. Word length grows with D (previous paragraph), so no finite set of words covers all starting values: the affine pieces are countably infinite. This — and nothing deeper — is the precise content of “ F has no closed form”: F is exactly computable, but only piece by piece, and the pieces never end. Equivalently, each cycle reads the binary expansion of D from the top: the opening R25 run consumes the leading bits (its length is the bit length, up to a constant), and the tail rows dispatch on what remains. Example: on the dominant word $w = (\text{R25 repeated } n \text{ times, then R23})$ — 1,096 of the first 3,000 cycles, the formula verified on each —

$$F|_w(D) = 16D - 240 \cdot 2^n + 32n + 169, \quad n = n_A(1, D).$$

6.3 The runner: the linear scan and the binary-search fix

Once per cascade, the runner must evaluate $n_A(b, d) = \max\{m \geq 1 : (b + 4) \cdot 2^{m+1} \leq d + 2(b + 4) + 2m\}$. The **linear scan** is the obvious algorithm: test $m = 1, 2, 3, \dots$ and stop at the first failure. It performs n_A comparisons, each touching integers as large as d , i.e. $\sim \text{bits}(d)$ bit operations per comparison — so one cascade costs about $n_A \cdot \text{bits}(D)$. On this orbit $n_A \approx \text{bits}(D) \approx 1.25k$ at cycle k (Section 6.2), so the scan costs $\sim (1.25k)^2$ per cycle and $\Theta(k^3)$ over k cycles; profiling showed this — not the arithmetic — dominated the level-3 deep run. The fix: the difference between the two sides of the guard inequality is strictly increasing in m , so the largest satisfying m can be found by **binary search** over $m \in \{1, \dots, \text{bits}(d) + 2\}$ — about $\log_2 \text{bits}(d)$ probes instead of n_A . (The same applies to n_B .) This removes the cubic term.

What remains is optimal up to constants. Representing D_k takes $\sim 1.25k$ bits (measured: 250,005 bits at $k = 200,000$), so merely *writing* the state of cycle k costs $\Theta(k)$ bit operations, and any simulator of k cycles must spend at least $\sum_{j \leq k} \Theta(j) = \Theta(k^2)$. The level-4 runner performs a handful of shift-and-add operations per cycle on numbers of exactly that size — it sits at this floor.

Runner	Time	Base steps covered (verified)
Level 3 runner	584 s	$10^{16,460}$ (300,000 macro steps)
Level 4 runner	10 s	same point, $\sim 60\times$ faster
Level 4, deep run	27 min	200,000 cycles: no halt in first $10^{150,514}$ steps

Verification. The binary-search round counts agree with the linear-scan versions on 3,000 random states per cascade, and F agrees with the composed level-3 steps — both resulting states and exact base-step counts — for the first 1,000 orbit cycles. The deep run checks the halting criterion of Section 8 at every level-2 macro state it passes, so the $10^{150,514}$ bound is certified, not sampled. Implementation: `onedim.py`.

6.4 The system after step 25

Fixing the start allows one final simplification: after exactly 25 base steps the machine is at $(0, 1, 0, 17)$, and from then on its entire future is the anchor dynamics of Section 6.1. The whole machine is therefore equivalent to the following two-variable system with initial state $(\mathbf{b}, \mathbf{d}) = (\mathbf{1}, \mathbf{17})$, **entered at base step 25**. The rules are those of the anchor map G (Section 6.1) with exactly two changes, marked as in Section 5.3: **cascadeA** — keeping its level-3 index R15 — replaces the round row R25 (all rounds re-batched, now with the interior-halt equation below), and **expand-deep** (R19) absorbs the boundary row R18 by widening its guard from $3d > 2b + 5$ to $3d \geq 2b + 5$ (merge proved below); every row marked • carries over from G unchanged, keeping its index. Guards remain mutually exclusive and order-free, halt last. The **Built from** column gives each row's origin in G, and each row carries its exact base-step certificate, with P and the “+ sweep” convention as in Section 6.1:

#	Name	Next state ($\mathbf{b}', \mathbf{d}'$)	Guard	Built from	Base steps
R16	recharge+ •	$(0, 2b + 4)$	$d = 0$ ($\dagger$)	R16	2
R7/8	seed •	$(2, 2b + 1); (2, 5)$ if $b = 0$	$d = 1$ ($\dagger$)	R7/8	3
R17	expand+ •	$(2d, 2b - 3d + 4)$	$d \geq 2, 3d < 2b + 5$	R17	$3d + 2$
R19	expand-deep (mod.)	$(6d - 4b - 8, 10b - 11d + 24)$	$d \leq b + 1, 3d \geq 2b + 5, 11d \leq 10b + 24$	R19, absorbs R18	$d + 3$
R20	expand-sweep •	$\text{SWEEP}(9d - 6b - 12, 8b - 8d + 20)$	$d \leq b + 1, 11d > 10b + 24$	R20	$2b - 2d + 7$ + sweep
R21	reset3+ •	$\text{SWEEP}(3b + 2, 6)$	$d = b + 3$	R21	1 + sweep
R22	reset4+ •	$\text{SWEEP}(3b + 4, 4)$	$d = b + 4$ ($\dagger$)	R22	1 + sweep
R23	exit •	$(1, 16(d - b) - 55)$	$b + 5 \leq d \leq 2b + 5, 15d > 18b + 61$	R23	$b + 5 +$ $P(4b - 2d + 12, 5d - 6b - 20)$
R24	shrink-sweep •	$\text{SWEEP}(6b - 3d + 19, 4d - 4b - 14)$	$b + 5 \leq d \leq 2b + 5, 15d \leq 18b + 61$	R24	$d - b - 3$ + sweep
R15	cascadeA (mod.)	$(2^n(b + 4) - 4, d - 2(b + 4)(2^n - 1) + 2n),$ $n = n_A(b, d)$	$d \geq 2b + 6$	$R25^n = (R12; R3)^n$	$(b + 4)(2^n - 1) - 2n$
R13	halt •	HALT	$d = b + 2$	R13	—

The cascade rule halts internally iff $3(b + 4) \cdot 2^i = d + 2b + 10 + 2i$ has an integer solution $0 \leq i \leq n$ (the round- i anchor then has $d = b + 2$); at anchor granularity this single equation subsumes both interior halting lines of Section 8.2, since the shrink-line value ψ_i equals ϕ_{i+1} . The boundary case $3d = 2b + 5$

(R18) needs no rule of its own: R19's remaining conjunct always holds there ($11d \leq 15d - 1$) and R19's formula specializes to $(2, 4d - 1)$ with the same cost $d + 3$, even though the underlying rule path differs — so the system has 11 rules. Rows marked (†) never fired in a 20,000-cycle census (usage: cascadeA 41,362; expand+ 29,382; exit 17,785; expand-deep 7,893; shrink-sweep 5,478; expand-sweep 2,605; reset3+ exactly once — the orbit's closest approach to halting), but they are near-diagonal cases of the same nature as HALT itself and equivalence requires keeping them. Verification (afterstep25.py): states and exact costs against composed level-2 steps on 40,000 random anchors, the boundary merge on 3,000 boundary states, and the orbit's D-sequence with exact cumulative base-step totals (including the +25 offset) reproduced for 1,000 cycles.

This system is at the computational floor. Per cycle it performs a handful of anchor steps (median 3, mean 4.6, maximum 42 across 20,000 cycles; expand chains have length ≤ 8 , so batching them — the expand map is linear with eigenvalues 1 and -4 — would save under 5%), and every state update is shift-and-add, linear in the bit length of D ; the exact-cost certificates add no measurable time at this scale (4.4 s vs 4.5 s for 20,000 cycles). Since D_k occupies $\sim 1.25k$ bits, merely writing the state of cycle k costs $\Theta(k)$ bit operations and simulating k cycles costs $\Theta(k^2)$ — which this system already achieves. The only conceivable “level 5” would be jumping straight to $F^k(17)$ without iterating, i.e. a closed form across branch words — and that is precisely the open Collatz-type core of Section 8.4. In other words: **further acceleration of this system is the open problem.** The acceleration hierarchy terminates here not for lack of effort but because its next rung and the halting question have become the same mathematical object.

Part II — The halting question, and why the machine is a cryptid

With the machine reduced to a one-integer map, its halting from $(0, 0, 0, 0)$ becomes a sharp question about a single orbit. We answer everything answerable — a halting criterion, an exact reduction, a verified non-halting bound, a mod-16 confinement — and locate precisely the one question that remains open, and why.

7. Qualitative dynamics

From almost every initial configuration the system never halts: each super-cycle (pump $\rightarrow$ cascadeA $\rightarrow$ sweep $\rightarrow$ pump ...) multiplies d by roughly a constant factor, so the state grows geometrically forever. The HALT rule (R13) is reachable only from a thin set of initial conditions — for example $(5, 0, 0, 7)$, which halts after 6 steps at $(0, 3, 0, 5)$. This unbounded growth is precisely why acceleration is essential: 3,000 level-2 macro steps from $(0, 0, 0, 0)$ already cover about 2.95×10^{35} base steps.

8. Does it halt from $(0, 0, 0, 0)$?

Answer: no halt within the first $10^{150,514}$ base steps (exact count verified), and almost certainly never — but a complete non-halting proof reduces to a Collatz-type problem that remains open. This section states precisely what is proved and what is heuristic.

8.1 An exact halting criterion

Define, on level-2 macro states,

$$\phi(a, b, d) = d - b - a.$$

Theorem. From any $c = 0$ configuration, the system halts if and only if its trajectory contains a macro state with $\phi = 2$ and $a \bmod 3 \neq 1$; such a state halts within at most 2 further macro steps.

Proof. ($\Leftarrow$) If $\phi = 2$ and $a = 0$ then $d = b + 2 \geq 2$: the halt state itself. If $a = 2$ then $d = b + 4 > 0$ and the two rule yields $(0, b + 1, b + 3)$: a halt state. If $a \geq 3$ then $d = a + b + 2 > \text{floor}(a/3)$, so the drain runs fully, $k = \text{floor}(a/3)$, to $(a \bmod 3, b + 2k, d - k)$; ϕ is invariant under the drain (each R3 firing changes d, b, a by $(-1, +2, -3)$, so $\Delta\phi = -1 - 2 + 3 = 0$), and $a \bmod 3 \in \{0, 2\}$ reduces to the previous cases. ($\Rightarrow$) A halt state has $\phi = 2$ and $a = 0$. The only rules that can output an $a = 0$ state with $\phi = 2$ are the drain (from $a \equiv 0 \bmod 3$) and the two rule — the other producers of $a = 0$ states miss $\phi = 2$ identically (jump gives $\phi = d + 2 \geq 3$; seed gives ϕ odd). Both preserve ϕ and have $a \bmod 3 \neq 1$, so walking back through the maximal drain/two suffix reaches a macro state satisfying the criterion. **QED**

The theorem was machine-checked against direct base-rule simulation on 10,932 states (an exhaustive box, random states, and adversarial states on and adjacent to the line $\phi = 2$), with horizon-exact accounting: 1,467 of them halt, and each halted at precisely the base step certified by the criterion — zero violations.

8.2 Corollary: four explicit halting lines

Checking which rules can *create* $\phi = 2$ (pump gives $\phi = 2b + 4d \geq 6$; jump ≥ 3 ; seed odd; the two resets give $4 - 3b$ and $-3b$, never 2), the system halts iff it ever reaches

$$a = 0 \text{ with } d = b + 2, \quad a = 0 \text{ with } d = 4b + 12, \quad a = 0 \text{ with } 5d = 2b + 2 \ (d \geq 2), \quad d = 0 \text{ with } a = 2b - 1, \ b \bmod 3 \neq 2.$$

This four-line characterization reproduced direct halting behavior exactly for every start $(0, 1, D)$, $D \leq 20,000$.

8.3 Reduction to a Collatz-type problem

By the unconditional-reduction proposition of Section 6.1, halting from $(0, 0, 0, 0)$ is equivalent to: *does the F -orbit of 17 (the sequence $D_0 = 17, D_{k+1} = F(D_k)$ of Section 6) ever enter the halting set $H = \{2, 3, 9, 16, 19, 26, 28, 39, 44, 49, 92, 102, 161, \dots\}$ of values D for which the super-cycle from $(0, 1, D)$ halts? H is sparse: $|H \cap [1, 10^6]| = 57$, roughly 10 elements per decade (density $\sim 4/x$). No congruence obstruction exists — the orbit visits every **residue class** of $d - b$ modulo 4 and 12 (that is, $d - b$ attains every possible remainder on division by 4 and by 12) — so no simple invariant can separate the orbit from H . This is precisely the structure of Collatz-type open problems: a deterministic orbit growing geometrically must forever avoid a sparse exceptional set.*

Theorem (H is infinite). For every $i \geq 0$ the machine halts from $(0, 1, 0, D)$ with $D = 15 \cdot 2^i - 2i - 12$; hence $H \supseteq \{3, 16, 44, 102, 220, 458, 936, 1894, \dots\}$ is infinite. *Proof.* Cascade A from $(0, 1, D)$ has round- i anchor (b_i, d_i) with $d_i - b_i = D + 12 + 2i - 15 \cdot 2^i$ (Section 5.1 closed forms with $b + 4 = 5$), which equals 2 exactly for this D — and an anchor with $d = b + 2$ is the halt state. Round i is reached because $n_A(1, D) \geq i$ reduces to $10 \cdot 2^i \leq 15 \cdot 2^i - 2$, true for all $i \geq 0$. **QED** (Machine-verified for $i = 0..29$). This explains the observed geometric spacing of H , and rules out settling non-halting by any finite

computation: the orbit must avoid an infinite, geometrically spaced set forever. More generally, every halting branch word w contributes the solution set of one explicit affine equation in $(D, 2^n, n)$ — H is exactly the countable union of these explicit families.

Theorem (mod-16 confinement). For every non-halting D , $F(D) \equiv 9 \pmod{16}$; hence $D_k \equiv 9 \pmod{16}$ for every $k \geq 1$. The first statement is about the map itself — whatever integer is fed in, F returns one that leaves remainder 9 on division by 16 — and the second is its consequence for the orbit, since every D_k with $k \geq 1$ is an output of F . (The seed $D_0 = 17 \equiv 1$ is the sole exception, as it is not itself an F -output; from there 57, 185, 601, 2265, ... are all $\equiv 9$.) *Proof.* $F(D)$ is the d -value of the $b = 1$ anchor closing the super-cycle, and a $b = 1$ anchor is produced only by exit (R23), giving $16(d - b) - 55 \equiv 9 \pmod{16}$ identically, or by SWEEP's pump-exit, giving $4\Delta^* + 1$ with $\Delta^* = (w \cdot 4^{n-1} - 2)/3$, $w = 12\Delta + 8$; since $\Delta^* \equiv 2 \pmod{4}$ in every case (for $n = 1$, $\Delta^* = 4\Delta + 2$; for $n \geq 2$, $w \cdot 4^{n-1} \equiv 0 \pmod{16}$ forces it), this too is $\equiv 9 \pmod{16}$. **QED** (Machine-verified universally.)

What this buys, and what it does not. Recall that H is a countable union of families, one per halting branch word (previous theorem). The orbit lives in the single class 9 mod 16, so it can only ever hit the part of H lying in that class. Two successive filters cut H down to that part, and it helps to see them separately.

Filter 1 — parity. The class 9 mod 16 is odd, but H is mostly even: of the 57 members below 1.5×10^6 , 45 are even and only 12 are odd. Every even member is unreachable at a stroke. This already disposes of the very family that proved H infinite: in $15 \cdot 2^i - 2i - 12$ every term with $i \geq 1$ is even, and the lone odd term ($i = 0$, giving $D = 3$) is $\equiv 3$, not 9. So the infinitude established in Section 8.3 lives entirely off the orbit — it says nothing about how many reachable targets exist.

Filter 2 — residue. The 12 odd members spread across five odd classes mod 16 — residues 1, 3, 7, 9, 15 — and the orbit is pinned to just one of them, 9. What survives both filters is $H \cap \{9 \pmod{16}\}$, which below 1.2×10^7 is only {9, 124313}; and since the orbit merely grows from 17, even 9 is behind it, so 124313 is the first value it could conceivably hit. Net effect: of every 57 elements of H , about 2 are reachable — a roughly thirty-fold thinning of the target set, and correspondingly of the heuristic hitting-probability in Section 8.4.

This is the tightest confinement a congruence can give: a single residue mod 16. Yet it settles nothing, for two honest reasons. First, H still meets the class — $124313 \in H \cap \{9 \pmod{16}\}$ — so the orbit is not separated from H ; and an exhaustive search confirms no modulus $m \leq 256$ separates them either (mod16.py). Second, whether the reachable set $H \cap \{9 \pmod{16}\}$ is even infinite is itself open: the Section 8.3 family is the wrong parity to help, no odd family $\equiv 9 \pmod{16}$ is known to be infinite, and only two elements have ever been found. Heuristically it should be infinite (each occupied odd class ought to receive its share), which is why the confinement strengthens the non-halting heuristic without proving it — but were it ever shown *finite*, checking the growing orbit against a finite list would settle non-halting outright. That we cannot decide even this is a miniature of the whole problem.

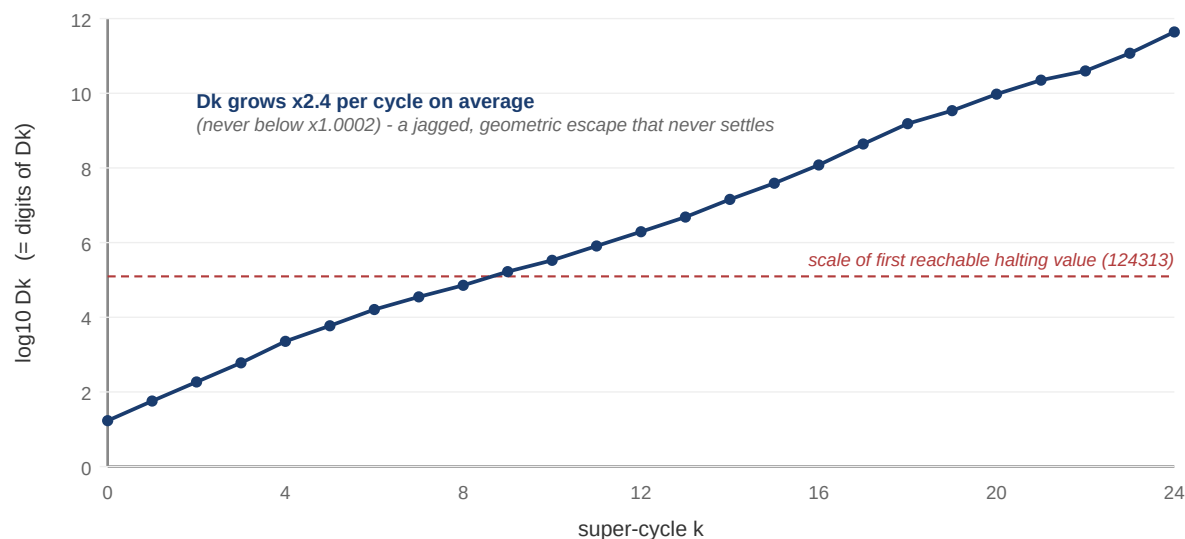


Figure 4. The F-orbit of 17 ($D_0 = 17$, $D_{k+1} = F(D_k)$), plotted as $\log_{10} D_k$ — the number of digits — against the cycle index k . The climb is roughly linear (geometric growth) but visibly jagged, the fingerprint of the pseudorandom branch words of Section 6.2. Halting would require the orbit to land *exactly* on a value in H ; the dashed line marks the scale of the first reachable such value, 124313, which the orbit sails past — passing a target’s magnitude is not hitting it.

8.4 Verified bound and heuristic conclusion

Checking the criterion at every level-2 macro state (including all states internal to cascades, located by exact binary search on the monotone ϕ -sequences), the level-3 runner verified no halt within $10^{16,460}$ base steps, and the level-4 runner extended this to **200,000 full super-cycles: the original system does not halt within its first $10^{150,514}$ steps** (the exact step count, a 150,515-digit number, is computed by the certificates; D then has 75,259 digits). Heuristically, cycle k has chance $\sim 4/D_k$ of landing in H ; summed over the surviving orbit this leaves total remaining halting “probability” below $10^{-75,000}$. The honest summary: **the system almost certainly never halts, this is verified beyond any physically simulable horizon, and a complete proof would require solving a Collatz-type orbit-avoidance problem.**

Why no proof is available in either direction. A halting proof would exhibit k with $D_k \in H$; none exists for $k \leq 200,000$ and the heuristic expectation is that none exists at all. A non-halting proof is blocked by three machine-checked facts: (i) no affine congruence invariant exists (symbolic check over the 12 non-halting base rules: only the trivial one survives); (ii) no residue class separates the orbit from H — and this is now known sharply, not just for a few moduli: the orbit is confined to the single tightest class 9 mod 16 (the mod-16 confinement theorem of Section 8.3), yet an exhaustive search of every modulus $m \leq 256$ finds H meeting the orbit’s residues at all of them, so even the best congruence still fails to separate; (iii) growth arguments cannot help: $F(D) > D$ empirically for every non-halting $D \leq 300,000$ (minimum ratio 1.0002), but since H is provably infinite with geometric spacing (Section 8.3), the growing orbit passes through H ’s scale range forever. Membership of D_k in H is an exact equality between D_k and closed forms such as $15 \cdot 2^i - 2i - 12$ — deciding it for all k requires bit-level control of the orbit of a piecewise-affine expanding map, which is precisely the open core of Collatz-type problems. The minimal open question is: *does the F-orbit of 17 avoid the countable union of explicit affine families that constitute H ?*

9. Why this machine is a cryptid

The preceding sections answered every question about this machine that can currently be answered. This section explains the nature of the one that remains: why it is not merely unsolved, but belongs to a class of problems that mathematics has so far been unable to touch — and how the acceleration program of Sections 3–6 is exactly what brought that fact into view.

9.1 What “Collatz-like” means

The Collatz map sends n to $n/2$ when n is even and to $3n + 1$ when n is odd. The conjecture — that every positive start eventually reaches 1 — dates to 1937, is numerically verified far beyond 10^{20} , and remains open; Erdős remarked that “mathematics may not be ready for such problems.” What makes it hard is not the specific constants but three structural features, and a single-orbit question deserves the name **Collatz-like** when it has all three:

(i) **Piecewise-affine dynamics**: finitely many affine branches, selected by simple conditions on the current value. Here: every G-case is affine (Section 6.2). (ii) **Expansion**: the map grows its argument on average, so orbits never revisit and never settle. Here: F grows D by a factor of 2.4 per cycle on average (and never shrinks it: minimum factor 1.0002 in 20,000 measured cycles). (iii) **Digit consumption**: which branch fires next depends on ever-deeper digits of the value, so the branch sequence behaves like a pseudo-random stream and no bounded summary of the state determines the future. Here: one cycle’s branch word has length $\sim \log_2 D$, at least 865 distinct words occur in the first 3,000 cycles, and the affine piece $F|_w$ is indexed by the entire word (Section 6.2, where words are defined). Feature (iii) is the fatal one: it is why no congruence, no automaton, and no invariant of bounded size can track the orbit.

This class is not merely mysterious; it is provably beyond general methods. Conway showed in 1972 that generalized Collatz maps — exactly the piecewise-affine, condition-dispatched maps above — can simulate arbitrary computation, so their termination problem is **undecidable in general**. No algorithm decides all instances. Every instance ever solved was solved by an instance-specific insight — an invariant, a potential function, a finite-state abstraction. Sections 8.3–8.4 record the systematic search for such an insight here, and its outcome: none of the known kinds exists for this machine.

9.2 The pseudorandom heuristic, and why it is only a heuristic

Feature (iii) is worth developing carefully, because it is the engine behind both why these problems are believed and why they resist proof. A deterministic sequence is called **pseudorandom** for a class of statistical tests if its statistics — frequencies, correlations, digit distributions — match those an independent random sequence would produce, even though the sequence is fully determined. The notion is always relative to a class of tests: no fixed deterministic sequence is “random”, yet many pass every test one can devise. The distinction that matters below is between a *proved* equidistribution statement (a theorem that certain averages converge to the random prediction) and an *unproved* hypothesis that a single prescribed orbit obeys those averages.

The Collatz heuristic, precisely. Restrict the map to odd n via the shortcut $T(n) = (3n + 1)/2^v$, where 2^v exactly divides $3n + 1$. Model the exponent v as geometric — $P(v = k) = 2^{-k}$ for $k \geq 1$, the prediction if the higher bits of $3n + 1$ were fair coin flips — so that $E[v] = 2$. The expected multiplicative change per odd step is then

$$E[\log(T(n)/n)] \approx \log 3 - E[v] \log 2 = \log 3 - 2 \log 2 = \log(3/4) < 0,$$

a contraction by a factor 3/4: the heuristic predicts almost every orbit drifts downward and hence reaches 1. What is genuinely *proved* supports the model but stops short of the conjecture. The Collatz map extends to a measure-preserving, ergodic transformation of the 2-adic integers, so its parity sequence is equidistributed for almost every start; and Tao (2019) showed that almost all orbits (in logarithmic density) attain almost bounded values. Both are **almost-everywhere** statements. The conjecture asks about the orbit of every n — each a single point, a measure-zero set — and an almost-everywhere theorem says nothing about any prescribed point. That gap between “almost every orbit” and “this orbit” is the whole of what remains open.

The same heuristic here. Our map F grows rather than shrinks: its average log-drift is $\log 2.4 \approx 0.88 > 0$, so the orbit almost surely escapes to infinity. The live question is therefore not descent but sparse hitting — does the escaping orbit ever land in H ? The pseudorandom hypothesis supplies the estimate: if the arithmetic of D_k that decides H -membership equidistributes, then $P(D_k \in H)$ is the local density of H at scale D_k . That density is measurable: fitting the counts ($|H \cap [1, 10^6]| = 57$) gives $|H \cap [1, x]| \approx 4.1 \ln x$, i.e. a local density $\approx 4/D_k$. The expected number of hits from cycle k onward is then $\sum_{j \geq k} 4/D_j$; since the D_j grow geometrically on average, the sum is controlled by its first term $4/D_k$. Past the verified horizon $k = 200,000$, where D_k already has 75,259 digits, the whole remaining sum is below $10^{-75,000}$ — the figure quoted in Section 8.4. The model’s verdict: the orbit almost surely never meets H , and the machine never halts.

Why the sequence is pseudorandom here — the mechanism. The structural reason is proved, in Section 6.2: F reads the binary expansion of D from the top. Each super-cycle’s opening R25 run consumes the leading $\sim \log_2 D$ bits, and the tail rows dispatch on the digits that remain; so the branch word is literally a function of the digit string of D , and advancing the orbit is an intricate digit-mixing operation — exactly as the Collatz parity sequence is a readout of the 2-adic digits of n . That is why the branch sequence carries no exploitable regularity, and the measurements confirm it: at least 865 distinct branch words in the first 3,000 cycles, anchors hitting every residue class of $d - b$ modulo 4 and 12, and a symbolic search returning only the trivial affine invariant (Section 8.4). These are precisely the statistics an equidistributed sequence would produce — the operational content of “pseudorandom” — and precisely why no invariant of bounded size can track the orbit.

The unbridgeable step. One honest asymmetry sharpens the difficulty. For Collatz the pseudorandom backbone is partly a theorem (2-adic ergodicity, Tao’s density-one result); for this machine we have no such theorem — no proved ergodicity for F — only the empirical signatures above. So the machine offers *less* rigorous structure than Collatz, not more, while exhibiting the same behavior. In both cases the final step is identical and unbridged: the heuristic is an assumption about one specific orbit, and every rigorous equidistribution result is almost-everywhere, silent on any single trajectory. The heuristic makes the answer (never halts) morally certain and even quantifies the certainty (below $10^{-75,000}$); it cannot make it a theorem, because turning “almost every orbit” into “this orbit” is exactly the open problem — here as for Collatz.

9.3 Cryptids

The term **cryptid** comes from the Busy Beaver community (bbchallenge.org), which classifies small Turing machines by whether their halting can be decided. The fifth Busy Beaver number was settled in 2024 only after the last holdout machines were resolved by bespoke proofs; at six states there

remain explicit machines — the most famous is *Antihydra*, which iterates $n \rightarrow 3n/2$ with a parity side-count — whose halting is equivalent to an open Collatz-type orbit question. Such machines are called cryptids: **small, fully explicit programs whose halting question is an open problem of Collatz type**. They matter because they mark the exact frontier where program size meets unprovability.

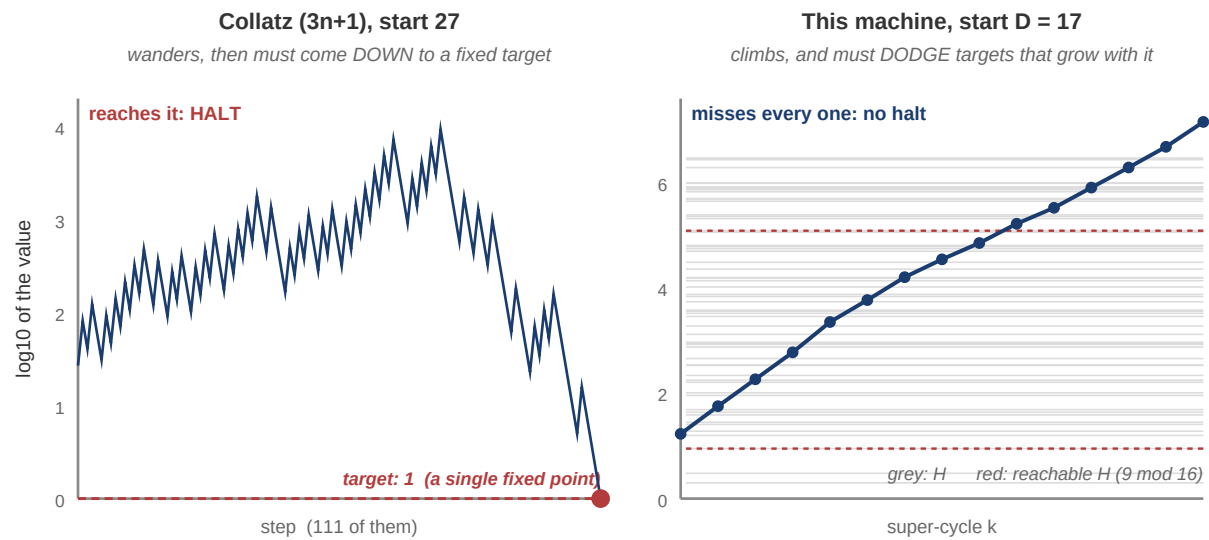
This machine is a cryptid in precisely that sense. It has 13 rules over four counters; every step is elementary arithmetic; and by the reduction of Sections 6 and 8 — unconditional, by the proposition of Section 6.1 — halting from $(0, 0, 0, 0)$ is *equivalent* to “the F-orbit of 17 avoids the halting set H” — with F expanding, piecewise affine with countably many digit-indexed pieces, and H an infinite, geometrically spaced union of explicit affine families. That claim deserves auditing rather than asserting. Each row below names one thing that makes this family of problems hard, what is actually *known* for Collatz, and what we were able to establish here:

what makes these problems hard	Collatz — what is known	This machine — what we established
Piecewise-affine dynamics	two affine branches, picked by a single parity bit.	every anchor rule is affine, and F is affine on each branch word — but with countably many pieces, not two (Section 6.2).
Expansion: orbits never settle	contracts by $\times 3/4$ per odd step on the standard heuristic, which is why it is expected to fall to 1; orbits are famously non-monotone.	expands and provably never shrinks on the dominant word : $F(D) - D \geq 2n + 19$ (Section 9.6) — a monotonicity Collatz does not have.
Digit consumption → pseudorandom branching	the branch is the next 2-adic digit of n ; the parity stream is provably equidistributed on the 2-adic integers.	each cycle consumes the $\sim \log_2 D$ leading bits; at least 865 distinct branch words in 3,000 cycles, every residue class visited (measured, not proved — Section 9.2).
Belongs to an undecidable class	Conway (1972): generalized Collatz maps simulate arbitrary computation, so termination is undecidable for the class — no general algorithm exists.	the same class, so only an instance-specific insight could ever settle it; Sections 8.3–8.4 record the search for one, and its failure.
The hard event is an exact coincidence	the orbit must reach exactly 1.	the orbit must land exactly on a member of H — an explicit affine family such as $15 \cdot 2^i - 2i - 12$.
Statistical theorems cannot reach it	Tao (2019): almost all orbits attain almost bounded values (logarithmic density one) — yet this says nothing about any single n .	the pseudorandom estimate puts the remaining chance below $10^{-75,000}$ — yet says nothing about this orbit (Section 9.5a).
No bounded-state invariant survives	no congruence or finite-automaton invariant is known.	symbolic search returns only the trivial invariant; the orbit is <i>provably</i> confined to $9 \bmod 16$, yet no modulus $m \leq 256$ separates it from H (Section 9.5b).
Growth arguments cannot close it	not applicable — the map contracts; the difficulty runs the other way.	escaping to infinity does not help: H is infinite with the same geometric spacing, so the orbit climbs through targets forever (Section 9.5c).

what makes these problems hard	Collatz — what is known	This machine — what we established
Verified far, proved not at all	checked far beyond 10^{20} ; open since 1937.	no halt in the first $10^{150,514}$ steps; open.
Proved structural backbone	2-adic ergodicity, plus Tao's density-one theorem — the branch is $n \bmod 2$, so the map lives on $\mathbb{Z}_2$.	provably unavailable by that route: the branch is a leading-digit quantity, and F has no continuous extension to $\mathbb{Z}_2$ at all (Section 9.6).

Read down the table, the verdict is not close: every structural feature that makes Collatz hard is present here, every avenue that fails for Collatz fails here for the same reason, and the last row runs the wrong way — Collatz at least has a proved ergodic backbone, and for this machine that route is provably closed (Section 9.6). A tiny, fully explicit program whose halting is equivalent to an open orbit-avoidance problem of exactly this kind is what the word **cryptid** names.

One difference is worth drawing out, because it inverts the question rather than restating it. Collatz sets a *wandering* orbit against a single *fixed* target and asks whether it ever arrives. This machine sets a *growing* orbit against a target set that *grows with it* and asks whether it forever fails to arrive. Both are open; the shapes are opposite:



Collatz asks whether a wandering orbit HITS one fixed point; this machine asks whether a growing orbit MISSES an infinite, growing set - forever

Figure 5. Two Collatz-type questions of opposite shape. *Left:* the Collatz orbit of 27 wanders for 111 steps and must come down to the single fixed target 1 — it does. *Right:* this machine climbs away geometrically through a field of halting values that itself extends to every scale (grey; the two reachable 9-mod-16 targets in red), and halting would require landing exactly on one. Passing a target's magnitude is not hitting it — which is why the verified bound of Section 8.4 settles nothing.

9.4 How the acceleration program exposed the core

The route matters, because it is itself the evidence. The hierarchy of Sections 3–6 acted as a lens: each level removed structure that was provably mechanical. Level 1 removed bookkeeping (the transfer buffer c). Level 2 removed arithmetic progressions (the unary loops R3 and R5, collapsed by

their own regularity). Level 3 removed geometric series (the two-rule cascades). Level 4 removed everything except one integer. At no point was anything difficult encountered — every reduction was exact, certified, and machine-verified; at every level the machine looked tame.

The hard core appeared at the precise moment nothing removable remained. When the whole machine had become the iteration of F on a single integer, the next question — does F itself have a finite closed form? — received a measured answer: no; its branch words grow without bound because F reads the binary expansion of D (Section 6.2). That is feature (iii) of Section 9.1 appearing exactly where all mechanical structure ran out — though, as Section 9.7 records, a first sign of it had already surfaced one level earlier, while merely defining F . The halting criterion (Section 8.1) then sharpened “does it halt” into hitting explicit affine lines; the H -is-infinite theorem (Section 8.3) closed the door on settling the question by any finite computation; and the three machine-checked obstructions (Section 8.4) closed the doors marked invariants, residues, and growth.

The terminal observation of Section 6.4 ties it together: the only conceivable further acceleration — a closed form for F^k across branch words — would be a solution to the halting problem. “This machine is a cryptid” and “the acceleration hierarchy terminates here” are the same fact seen from two sides. That suggests an operational definition worth stating: **accelerate a machine until only closed forms remain; what is left is its irreducible mathematical content. For a tame machine the residue is empty — the accelerations run to a full solution. A cryptid is a machine whose residue is an open problem.**

9.5 The anatomy of the hardness

(a) Exact events defeat statistical tools. The strongest modern result in this area — Tao’s 2019 theorem that almost all Collatz orbits attain almost bounded values — is a statement about density-one sets of starting points (statements true for 100% of starts in the limiting-fraction sense). Halting here demands an exact equality, $D_k = 15 \cdot 2^i - 2i - 12$ or a sibling, for one specific orbit: a measure-zero coincidence — an event a randomly drawn orbit would satisfy with probability zero. Statistical methods discard, by construction, exactly the information that decides the question.

(b) Digit consumption destroys invariants. For any congruence or finite-automaton invariant to exist, the branch taken must factor through a bounded amount of state. It does not: branch selection involves magnitude comparisons ($n_A \approx \log_2 D$ of them per cycle) that residues cannot express, the orbit’s anchors hit every residue class of $d - b$ modulo 4 and 12, and the symbolic search over affine invariants returns only the trivial one (the digit-mixing mechanism behind this is developed in Section 9.2). These are measurements (Section 8.4), not impressions.

(c) Growth cannot save. $F(D) > D$ on every non-halting tested start, so the orbit escapes to infinity — but H is provably infinite with the same geometric spacing as the orbit’s growth (Section 8.3): the orbit climbs *through* H ’s scale range forever, and escape arguments are structurally useless. This is exactly why the H -is-infinite theorem matters: it converts “we have not found a proof” into “whole categories of proof are impossible.”

(d) The heuristic–proof gap. By the pseudorandom estimate of Section 9.2 the expected number of future hits on H is below $10^{-75,000}$ — overwhelming certainty by any scientific standard. Yet for a deterministic orbit, probability zero is not impossibility. Collatz-like problems are precisely where these two notions diverge maximally: the orbit is random enough to make the answer morally

obvious, and deterministic enough to make that answer unprovable with current mathematics. That gap — verified to $10^{150,514}$ steps and provable by no known method — is what this report ultimately documents.

9.6 What can be proved after all

Two entries in the scorecard were, for most of this report, mere measurements where Collatz has theorems. Both can be improved — one into a proof, the other into an *obstruction* — a third, natural conjecture turns out to be false, and a fourth result — found later, prompted by the second machine in this collection — closes the cycle question outright. Verification: gaps.py for (a)–(c), potential.py for (d).

(a) Expansion is a theorem on the dominant word, with an exact margin. On $w = (R25^n, R23)$ the closed form of Section 6.2 gives $F(D) = 16D - 240 \cdot 2^n + 32n + 169$ with $n = n_A(1, D)$. Writing $D = 16 \cdot 2^n + \delta$,

$$F(D) - D = 15\delta + 32n + 169.$$

The exit row R23 fires only while its guard $15d > 18b + 61$ holds, and after n cascade-A rounds $b = 5 \cdot 2^n - 4$ and $d = 6 \cdot 2^n + \delta + 10 + 2n$, so that guard reads $15\delta + 30n + 161 > 0$. Substituting, $F(D) - D > (-30n - 161) + 32n + 169 = 2n + 8 > 0$; and since δ is an integer the minimum is attained at $\delta = -(2n + 10)$:

$$F(D) - D \geq 2n + 19, \quad \text{attained at } D = 16 \cdot 2^n - (2n + 10).$$

QED (verified for $n = 6..39$.) So the dominant word cannot contract — and only just: the guard fails exactly 11 units before the contraction region begins. This also explains the puzzling measurement the report carried earlier, that the minimum growth ratio creeps toward 1: the margin $2n + 19$ grows logarithmically while D grows exponentially. No contraction occurs anywhere else we can reach either (no $F(D) \leq D$ among $\sim 450,000$ values spanning eighteen orders of magnitude, plus a systematic probe of the boundaries $c \cdot 2^n$). Note this is a monotonicity *Collatz does not have*: its orbits go up and down, while this one provably climbs.

(b) The missing backbone is not absent but obstructed. Collatz's 2-adic ergodicity exists for a reason: its branch is chosen by $n \bmod 2$ — low-order, 2-adic data — so the map extends continuously to the 2-adic integers and carries an ergodic measure. This machine's branch count is $n_A(1, D) = \text{bits}(D) - 4 + \varepsilon$, with $\varepsilon \in \{-1, 0\}$ fixed by the leading digits: an **Archimedean** quantity, not a 2-adic one. Agreeing on low-order bits therefore says nothing about which branch fires, and continuity fails at every level.

Theorem. For every N there exist $D \equiv D' \pmod{2^N}$ with $F(D) \not\equiv F(D') \pmod{32}$. Hence F admits no continuous extension to $\mathbb{Z}_2$. *Proof (construction).* Near $\text{base} = 16 \cdot 2^n$ the exit guard flips at $\delta = -(2n + 10)$, by (a). Take

$$D' = \text{base} - (2n + 11) \text{ (the word has switched), } D = D' + 2^N \text{ (back on the dominant word).}$$

They agree $\bmod 2^N$ by construction, lie in different branch words, and give $F \equiv 25$ and $9 \pmod{32}$ respectively. **QED** (constructed and checked for N up to 2^{128} .) The consequence is worth stating plainly: **the Collatz 2-adic argument provably cannot be transplanted here.** The last row of the scorecard is therefore not an admission of missing work but a structural fact about this machine.

(c) The natural replacement conjecture is false. If the branch is Archimedean, the right analogue of the 2-adic circle should be the **mantissa circle**, $\text{frac}(\log_2 D)$ — and the obvious conjecture is that the orbit equidistributes on it. It does not: over 20,000 cycles a 20-bin test gives $\chi^2 \approx 162$ against a 95% critical value of about 30. The mantissa instead carries a genuine non-uniform stationary density. So a backbone for F would have to identify *that* density; the naive equidistribution statement is already refuted. This is the most concrete target we can hand to a future attempt — and a reminder that the honest move, when a conjecture is cheap to test, is to test it.

(d) A monotone potential: the machine cannot cycle. The after-step-25 system of Section 6.4 admits an exact potential. Write $\Phi = 2b + d$. Then **every non-halting rule increases Φ by at least 2**. For the direct rules this is a computation: the cascade gives $\Delta\Phi = 2n$ exactly (its $(b+4)(2^n - 1)$ terms cancel two-for-one); the exit rule gives $\Delta\Phi = 15d - 18b - 53 \geq 9$, its guard $15d > 18b + 61$ being precisely the potential's positivity condition — the same guard-sits-just-above-zero phenomenon as the margin in (a); the expansion rules give +4 and +8 exactly. The sweep rules rest on a conservation lemma: if $\text{SWEEP}(A, \Delta)$ runs its cascade for n rounds, its pre-drain outputs satisfy $A^* + \Delta^* = A + \Delta + 3n$, and each drain case then lands within 3 of that sum (the pump case exceeds it). Finally, the guards of the two sweep-entry rules turn out to be *equivalent* to their cascade running at least once, which supplies the needed +3n. Minimum increment over all rules: 2, attained by a one-round cascade — matching the measured minimum exactly.

Corollary. The system admits no cycles, and the trajectory from (0, 0, 0, 0) never repeats a state after step 25. This closes unconditionally the one scenario that no amount of simulation could exclude: an eventually-periodic orbit with an astronomically long transient. The scenario is not hypothetical — the Busy Beaver machine Skelet #1 looked chaotic yet proved to be a translated cyler with a preperiod near 5.4×10^{51} steps — and before this theorem, the report's no-cycling evidence was only the measured expansion ratio. Verified: the lemma on 60,000 cascades, the increment on ~150,000 transitions spanning ten orders of magnitude (minimum found: 2), and 300,000 anchor steps of the true orbit.

None of this touches the halting question, which remains exactly as open as before. What changes is the shape of the ignorance: one empirical claim became a theorem, one gap became a proved obstruction, one plausible route was closed by counterexample rather than left untried — and the cycling alternative is now excluded by proof, so the dichotomy of Section 8 is clean: the machine halts, or it escapes to infinity; it cannot wander periodically.

9.7 The hardness is nested

One structural fact, uncovered while formalizing Section 6, sharpens the whole picture: the halting question is not the only Collatz-type question this machine raises — it is the outermost of two.

The inner one appeared while merely *defining* F. Turning the anchor dynamics into a one-variable map needed the orbit to return to the section $\{b = 1\}$ each super-cycle (Section 6.1). That return is guaranteed by no theorem, and for a telling reason: in the ratio coordinate $x = d/b$ the tail rows act as an expanding interval map, and returning means eventually landing in fixed exit windows — so “returns forever” is precisely “an expanding orbit that never avoids a fixed target,” the same shape as a Collatz orbit avoiding a sparse set. The return question is Collatz-like in its own right, one level below the halting question that governs the orbit of super-cycles.

Two things set the inner question apart from the outer one, and together they are the reason the report can conclude anything at all. It is **dispensable**: the unconditional-reduction proposition (Section 6.1) shows a non-returning orbit would simply be one more non-halting run, so the inner question can be sidestepped without being solved — whereas the outer question cannot be sidestepped, only left open. And it is a **structural analogy**, not a proven equivalence: unlike the halting question, which reduces to an exact, explicitly described orbit-avoidance problem (Section 8), the return question merely has the same expanding-map-avoids-thin-set form.

So the machine is Collatz-hard not at a single isolated point but through a nested pair of same-shaped questions — one dispensable, one essential. This is the precise sense in which its difficulty is intrinsic rather than incidental, and it refines the operational picture of Section 9.4. Accelerating a machine does not always reduce it to one irreducible question; it can peel the machine into a stack of Collatz-type questions. What makes this machine a cryptid is not merely that the stack is non-empty, but that it does not bottom out at anything solvable: the essential layer is an open problem, and the acceleration hierarchy terminates there because the only way past it is to solve it.

10. Verification

All verification is against the original 13-rule system, executed literally, and all checks are step-exact: a macro rule claiming k base steps must land on the identical configuration after exactly k applications of the original rules. (“Checkpoints” below are the states an accelerated system visits, located at their certified positions inside the base trajectory.)

Check	Scope	Result
Per-rule exactness (level 2)	~49,600 states: exhaustive over all triples with values < 25 , plus random states with values up to 10^6 , covering every guard	pass
Whole-trajectory vs. base (level 2)	85 random/adversarial starts, 300,000 base steps each; every macro checkpoint matches the base state at the claimed step index	pass
Level 3 vs. level 2 checkpoints	60 starts, 4,000 macro steps each	pass
Closed-form cascades (Section 5.3)	round counts, states and costs vs. loop implementations: 50,000 random states per cascade; assembled system: 20,000 further states + trajectories	pass
Level 3 vs. base directly	41 starts including large states, up to 500,000 base steps each	pass
Halting behavior	halting starts (e.g. (5, 0, 0, 7)) halt at the same configuration after the same number of steps at every level	pass
Halting criterion (Section 8.1)	criterion vs. direct base simulation, horizon-exact: 10,932 states (exhaustive box + random + near-line adversarial), 1,467 halts each at the certified step; four-line corollary vs. direct halting on all (0, 1, D), $D \leq 20,000$	pass
Anchor map G (Section 6.1)	G vs. composed level-2 steps on 40,000 random anchors across ten orders of magnitude; F-as-first-return vs. the level-4 runner on 800 orbit cycles; the dominant affine piece on 1,096 cycles	pass

Check	Scope	Result
Cascade-internal line checks (Section 8.4)	binary-search checks vs. brute-force cascade expansion on 60,000 random states; 3,896 engineered on-line states all flagged	pass
Monotone potential (Section 9.6d)	conservation lemma $A^* + \Delta^* = A + \Delta + 3n$ on 60,000 cascades; guard-cascade equivalences for both sweep rules exhaustively to 2,000; $\Delta\Phi \geq 2$ on ~150,000 transitions (minimum 2) and 300,000 orbit anchor steps	pass

Erratum (found and fixed). Verifying the anchor map of Section 6.1 against the base rules exposed a bug in the level-3 implementation: the seed rule had been written as $(0, 2, \max(2b + 1, 5))$, which is wrong at exactly $b = 1$ (the correct successor of $(0, 1, 0, 1)$ is $(0, 2, 0, 3)$, confirmed against the original 13 rules). The error affects only the single state $(0, 1, 1)$ — a cycle boundary with $D = 1$, which the orbit of $D = 17$ never visits — and all code and tables have been corrected. The full 200,000-cycle deep run was re-executed with the corrected rule and reproduces the original run exactly (identical step-count exponents at every 50,000-cycle checkpoint and identical final D), so the $10^{150,514}$ bound of Section 8.4 stands as stated.

11. Performance

System	Steps to cover 10^{60} base steps	Range observed
Base system	10^{60}	— (infeasible)
Level 2	~8,700	~8,200–8,900 across starts
Level 3	~1,000	~980–1,030 across starts
Level 4 (from $(0,0,0,0)$)	~80 cycles	one cycle $\approx$ 10 rule applications; see Section 6

Because each accelerated step is constant-cost integer arithmetic and cascades cover exponentially many base steps, simulating N base steps requires only polylogarithmic work in N .

12. Code artifacts

<code>collatz.py</code>	the original 13-rule base system, exactly as specified
<code>accel.py</code>	level-2 accelerated system with per-rule step-exact verification
<code>accel2.py</code>	level-3 cascade super-rules, verified against level 2
<code>accel3.py</code>	the complete closed-form level-3 system of Section 5.3, verified
<code>criterion.py</code>	the halting criterion of Section 8.1, proof notes and full verification
<code>orbit.py</code>	the deep verified run from $(0, 0, 0, 0)$ with cascade-internal line checks
<code>onedim.py</code>	level 4: the one-dimensional return-map system for the fixed start, verified
<code>formal.py</code>	the anchor map G of Section 6.1 and the piecewise-affine structure of F , verified

<code>afterstep25.py</code>	the after-step-25 system of Section 6.4 with exact cost certificates, verified
<code>mod16.py</code>	the mod-16 confinement theorem of Section 8.3 and the modulus-search, verified
<code>gaps.py</code>	the three results of Section 9.6: the expansion margin, the 2-adic obstruction, and the mantissa disproof
<code>potential.py</code>	the monotone potential $\Phi = 2b + d$ of Section 9.6(d): conservation lemma, per-rule increments, no-cycle corollary
<code>traj.py</code>	whole-trajectory equivalence checks and rule-usage profiling

Each file re-runs its own verification suite when executed with `python3 <file>`.